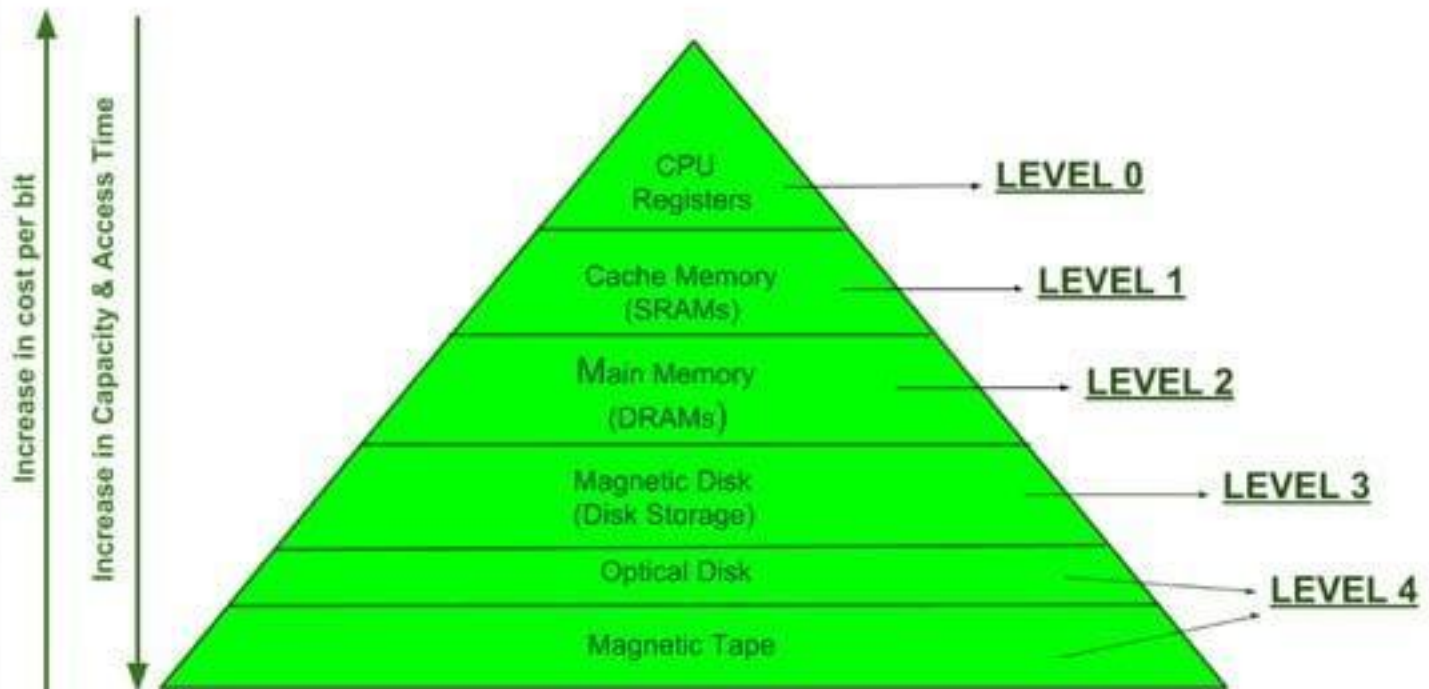


MEMORY HIERARCHY



MEMORY HIERARCHY DESIGN

In the design of the computer system, a processor, as well as a large amount of memory devices, has been used. However, the main problem is, these parts are expensive. So the memory organization of the system can be done by memory hierarchy. It has several levels of memory with different performance rates. But all these can supply an exact purpose, such that the access time can be reduced.

- Memory Hierarchy was developed depending upon the behaviour of the program.
- The Memory in the Computer are divided into five hierarchies based on the speed as well as uses.
- The processor can move from one level to another level based on its requirement
- The five hierarchies are:

- 1. Register (Volatile Memories)**
- 2. Cache (Volatile Memories)**
- 3. Main Memory (Volatile Memories)**
- 4. Magnetic Disk (Non-Volatile Memories)**
- 5. Magnetic Tape. (Non-Volatile Memories)**

- The first three hierarchies are volatile memories (Lose data completely when power off)
- Whereas the last 2 hierarchies are Not Volatile memories (Store data permanently)

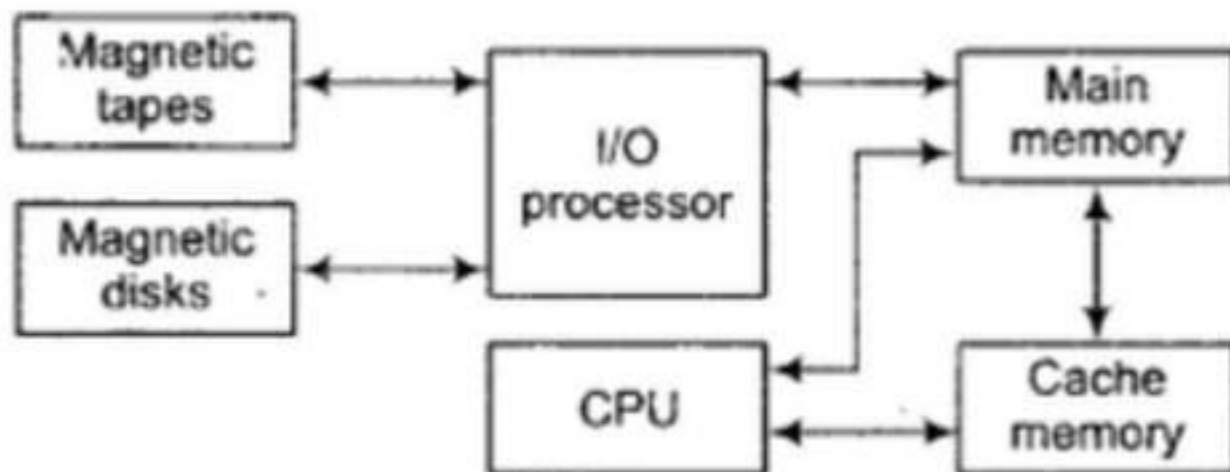
Memory Hierarchy Design is divided into 2 main types:

✓ **External Memory (Secondary Memory):**

Comprising of Magnetic Disk, Optical Disk, Magnetic Tape i.e. peripheral storage devices which are accessible by the processor via I/O Module.

✓ **Internal Memory (Primary Memory):-**

Comprising of Main Memory, Cache Memory & CPU registers. This is directly accessible by the processor.



MEMORY CONNECTION IN COMPUTER SYSTEM

REGISTERS:

Usually, the register is a static RAM or SRAM in the processor of the computer which is used for holding the data word which is typically 64 or 128 bits. The program counter register is the most important as well as found in all the processors. Most of the processors use a status word register as well as an accumulator. A status word register is used for decision making, and the accumulator is used to store the data like mathematical operation. Usually, computers like complex instruction set computers have so many registers for accepting main memory, and RISC- reduced instruction set computers have more registers.

CACHE MEMORY:

- A small, fast storage memory used to improve average access time Or We can say that cache is a very high-speed memory that is used to increase the speed of processing by making current programs and data available to the CPU at a rapid rate.
- The cache is used for storing segments of programs currently being executed in the CPU and temporary data frequently needed in the present calculations.

CACHE PERFORMANCE:

When the processor needs to read or write to a location in main memory, it first checks whether a copy of that data is in the cache. If so, the processor immediately reads from or writes to the cache.

Cache hit If the processor immediately reads or writes the data in the cache line.

Cache miss If the processor does not find the required word in cache, then cache miss has occurred.

MAIN MEMORY:

- The main memory refers to the physical memory and it is one central storage unit in a computer system.
- The main memory is relatively large and fast memory used to store programs and data during the computer operation.
- The main memory in a general-purpose computer is made up of RAM integrated circuit.

LATENCY:

The latency is the time taken to transfer a block of data either from main memory or caches.

- As the CPU executes instructions, both the instructions themselves and the data they operate on must be brought into the registers, until the instruction/data is available, the CPU cannot proceed to execute it and must wait. The latency is thus the time the CPU waits to obtain the data.
- The latency of the main memory directly influences the efficiency of the CPU.

AUXILIARY-MEMORY:

The common auxiliary memory devices used in computer systems are magnetic disks and tapes.

Magnetic Disks

- A magnetic disk, is a circular plate constructed of metal or plastic coated with magnetized material.
- Often, both sides of the disk are used and several disks may be stacked on one spindle with read/write heads available on each surface.
- All disks rotate together at high speed. Bits are stored in the magnetized surface in spots along concentric circles called tracks. The tracks are commonly divided into sections called sectors.

MAGNETIC TAPES:

- A magnetic tape is a medium of magnetic recording, made of a thin magnetizable coating on a long, narrow strip of plastic film.
- Bits are recorded as magnetic spots on the tape along several tracks. Magnetic tapes can be stopped, started to move forward or in reverse. Read/write heads are mounted one in each track, so that data can be recorded and read as a sequence of characters.

Characteristic of Memory Hierarchy:

✓ **Performance:**

One of the most significant ways to increase system performance is minimizing how far down the memory hierarchy one has to go to manipulate data.

✓ **Capacity:**

It is the global volume of information the memory can store. As we move from top to bottom in the Hierarchy, the capacity increases.

✓ **Access Time:**

It is the time interval between the read/write request and the availability of the data. As we move from top to bottom in the Hierarchy, the access time increases.

✓ **Cost per bit:**

As we move from bottom to top in the Hierarchy, the cost per bit increases i.e. Internal Memory is costlier than External Memory.

Advantages of Memory Hierarchy:

- ✓ Memory distributing is simple and economical
- ✓ Removes external destruction
- ✓ Data can be spread all over

- ✓ Permits demand paging & pre-paging
- ✓ Swapping will be more proficient

Inclusion, Coherence, and Locality Properties: ($M_1, M_2, M_3 \dots M_n$)

- ✓ Cache memory the innermost level M_1 , which directly communicates with the CPU registers.
- ✓ The outermost level M_n contains all the information words stored.
- ✓ In fact, the collection of all addressable words in M_n forms the virtual address space of a computer.

Inclusion Properties:

- ✓ The inclusion properties is stated as $M_1, M_2, M_3 \dots M_n$. The set inclusion relationship implies that all information items are originally stored in the outermost level M_n .
- ✓ During the processing, subsets of M_n are copied into M_{n-1} . Similarly, subsets of M_{n-1} are copied into M_{n-2} and so on.
- ✓ If an information word is found in M_i , then copies of the same word can also be found in all upper levels
- ✓ However, a word stored in upper level, may not be found in lower level in M_i

Coherence Property:

- ✓ The **Coherence** property requires that copies of the same information item at successive memory levels be consistent.
- ✓ If a word is modified in the cache, copies of that word must be updated immediately or eventually at all higher levels. The hierarchy should be maintained as such.
- ✓ Frequently used information is often found in the lower levels in order to minimize the effective access time of the memory hierarchy.
- ✓ In general, there are two strategies for maintaining the coherence in a memory hierarchy.
 - i. **Write-through (WT)** -Which demand immediate update in M_{n+1} if a word is modified in M_i .
 - ii. **Write-Back(WB)**- Which delays the update in M_{i+1} until the word being modified in M_i is replaced or removed from M_i .

Locality Property:

- ✓ Locality of reference refers to a phenomenon in which a computer program tends to access same set of memory locations for a particular time period.
- ✓ **Locality of Reference** refers to the tendency of the computer program to access instructions whose addresses are near one another.

- ✓ The property of locality of reference is mainly shown by loops and subroutine calls in a program.
 - i. In case of **loops** in program control processing unit repeatedly refers to the set of instructions that constitute the loop.
 - ii. In case of **subroutine** calls, every time the set of instructions are fetched from memory.
 - iii. References to data items also get localized that means same data item is referenced again and again.

Cache Operation:

It is based on the principle of **locality of reference**. There are two ways with which data or instruction is fetched from main memory and get stored in cache memory. These two ways are the following:

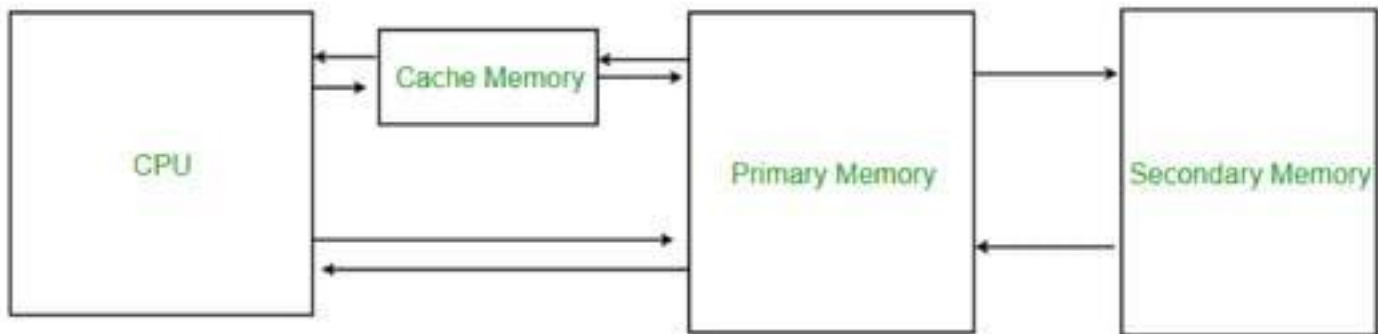
1. Temporal Locality:

- i. Temporal locality means current data or instruction that is being fetched may be needed soon. So we should store that data or instruction in the cache memory so that we can avoid again searching in main memory for the same data.
- ii. When CPU accesses the current main memory location for reading required data or instruction, it also gets stored in the cache memory which is based on the fact that same data or instruction may be needed in near future. This is known as temporal locality. If some data is referenced, then there is a high probability that it will be referenced again in the near future.

2. Spatial Locality:

- i. Spatial locality means instruction or data near to the current memory location that is being fetched, may be needed soon in the near future. This is slightly different from the temporal locality.
- ii. Here we are talking about nearly located memory locations while in temporal locality we were talking about the actual memory location that was being fetched.

CACHE MEMORY DESIGN AND IMPLEMENTATION



Cache Memory:

- ✓ It's a special very high-speed memory.
- ✓ It is used to speed up and synchronizing with high-speed CPU.
- ✓ Cache memory is costlier than main memory or disk memory but economical than CPU registers.
- ✓ Cache memory is an extremely fast memory type that acts as a buffer between RAM and the CPU.
- ✓ It holds frequently requested data and instructions so that they are immediately available to the CPU when needed.
- ✓ Cache memory is used to reduce the average time to access data from the Main memory.
- ✓ The cache is a smaller and faster memory which stores copies of the data from frequently used main memory locations. There are various different independent caches in a CPU, which store instructions and data.

CACHE PERFORMANCE:

- ✓ When the processor needs to read or write a location in main memory, it first checks for a corresponding entry in the cache.
 - If the processor finds that the memory location is in the cache, a **cache hit** has occurred and data is read from cache.
 - If the processor **does not** find the memory location in the cache, a **cache miss** has occurred. For a cache miss, the cache allocates a new entry and copies in data from main memory, then the request is fulfilled from the contents of the cache.
- ✓ The performance of cache memory is frequently measured in terms of a quantity called **Hit ratio**.

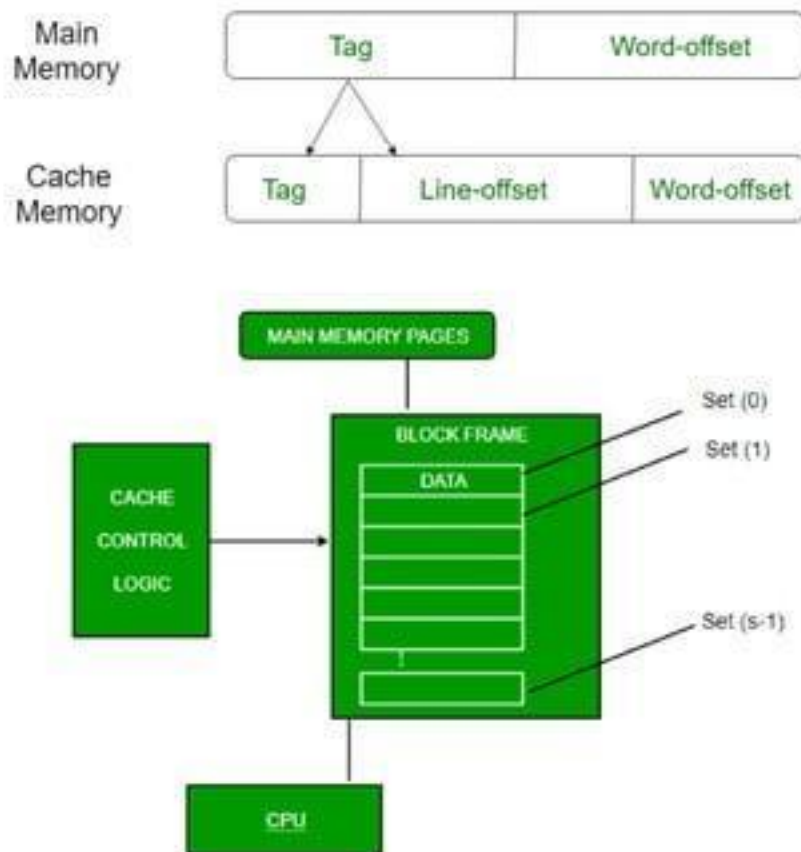
$$\text{Hit Ratio} = \frac{\text{hit}}{\text{hit} + \text{miss}} = \text{no. of hits / total accesses}$$

- ✓ We can improve Cache performance using higher cache block size, higher associativity, reduce miss rate, reduce miss penalty, and reduce the time to hit in the cache.

Cache Mapping:

There are **three different types** of mapping used for the purpose of **cache memory** which are as follows:

1. Direct Mapping –



- ✓ The simplest technique, known as direct mapping, maps each block of main memory into only one possible cache line. Or
- ✓ In Direct mapping, assigned each memory block to a specific line in the cache.
- ✓ If a line is previously taken up by a memory block when a new block needs to be loaded, the old block is trashed.
- ✓ An address space is split into two parts **index field** and a **tag field**.
- ✓ The cache is used to store the tag field whereas the rest is stored in the main memory.

- ✓ Direct mapping's performance is directly proportional to the Hit ratio.

$$i = j \text{ modulo } m$$

where

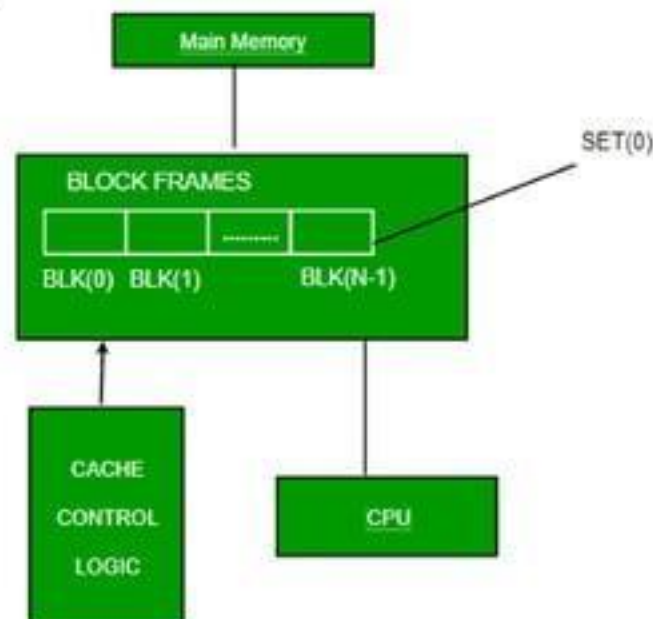
i = cache line number

j = main memory block number

m = number of lines in the cache

- ✓ For purposes of cache access, each main memory address can be viewed as consisting of three fields. The least significant w bits identify a unique word or byte within a block of main memory.
- ✓ In most contemporary machines, the address is at the byte level. The remaining s bits specify one of the 2^s blocks of main memory. The cache logic interprets these s bits as a tag of $s-r$ bits (most significant portion) and a line field of r bits. This latter field identifies one of the $m=2^r$ lines of the cache.

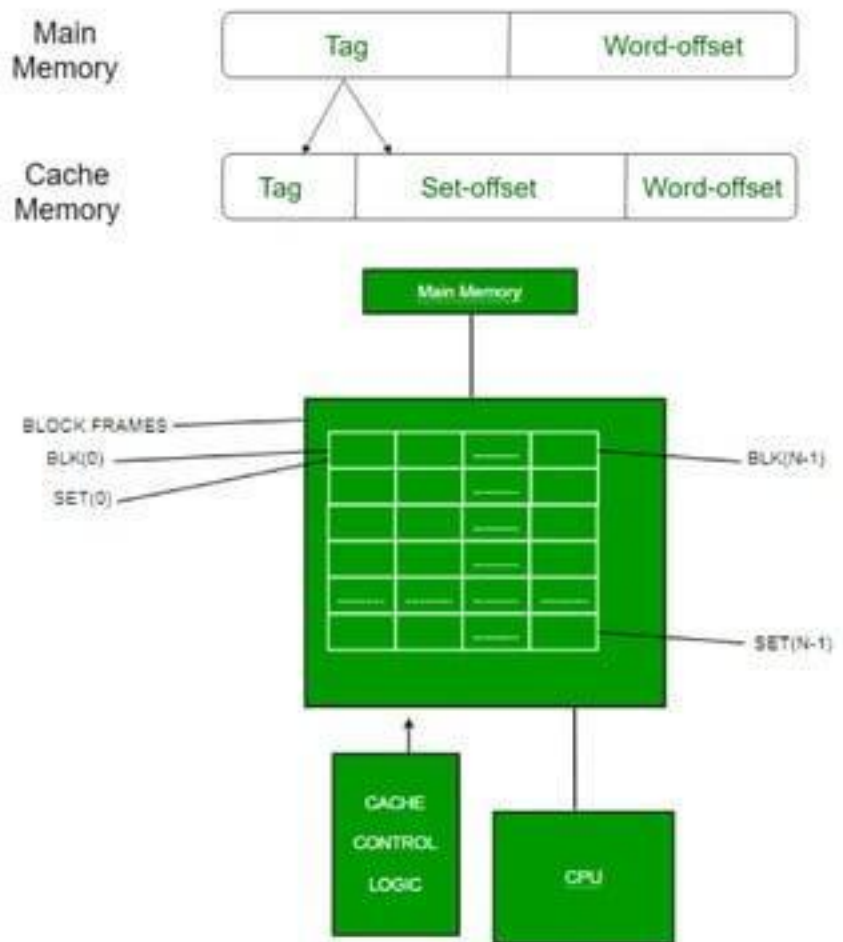
Associative Mapping -



- ✓ In this type of mapping, the associative memory is used to store **content** and **addresses** of the memory word.
- ✓ Any block can go into any line of the cache. This means that the word **id** bits are used to identify which word in the block is needed, but the tag becomes all of the remaining bits.
- ✓ This enables the placement of any word at any place in the cache memory.
- ✓ It is considered to be the fastest and the most flexible mapping form.

Set-associative Mapping –

- ✓ This form of mapping is an **enhanced** form of direct mapping where the drawbacks of direct mapping are removed.
- ✓ Set associative addresses the problem of possible thrashing in the direct mapping method.
- ✓ It does this by saying that instead of having exactly one line that a block can map to in the cache, we will group a few lines together creating a **set**.
- ✓ Then a block in memory can map to any one of the lines of a specific set.



- ✓ Set-associative mapping allows that each word that is present in the cache can have two or more words in the main memory for the same index address.
- ✓ Set associative cache mapping combines the best of direct and associative cache mapping techniques.
- ✓ In this case, the cache consists of a number of sets, each of which consists of a number of lines. The relationships are

$$m = v * k$$
$$i = j \bmod v$$

where

i=cache set number

j=main memory block number

v=number of sets

m=number of lines in the cache number of sets

k=number of lines in each set

APPLICATION OF CACHE MEMORY:

1. Usually, the cache memory can store a reasonable number of blocks at any given time, but this number is small compared to the total number of blocks in the main memory.
2. The correspondence between the main memory blocks and those in the cache is specified by a mapping function.

TYPES OF CACHE:

- **PRIMARY CACHE:**

A primary cache is always located on the processor chip. This cache is small and its access time is comparable to that of processor registers.

- **SECONDARY CACHE:**

Secondary cache is placed between the primary cache and the rest of the memory. It is referred to as the level 2 (L2) cache. Often, the Level 2 cache is also housed on the processor chip.

LOCALITY OF REFERENCE:

Since size of cache memory is less as compared to main memory. So to check which part of main memory should be given priority and loaded in cache is decided based on locality of reference.

TYPES OF LOCALITY OF REFERENCE:

1. **SPATIAL LOCALITY OF REFERENCE:**

This says that there is a chance that element will be present in the close proximity to the reference point and next time if again searched then more close proximity to the point of reference.

2. **TEMPORAL LOCALITY OF REFERENCE:**

In this Least recently used algorithm will be used. Whenever there is page fault occurs within a word will not only load word in main memory but complete page fault will be loaded because spatial locality of reference rule says that if you are referring any word next word will be referred in its register that's why we load complete page table so the complete block will be loaded.

TECHNIQUES TO REDUCE CACHE MISS PENALTY

- ✓ Cache is a random-access memory used by the CPU to reduce the average time taken to access memory.

1. FIRST MISS PENALTY REDUCTION TECHNIQUE:

(MULTI-LEVEL CACHES)

- ✓ **Multilevel Caches** is one of the techniques to improve Cache Performance by reducing the "**MISS PENALTY**".
- ✓ Miss Penalty refers to the extra time required to bring the data into cache from the Main memory whenever there is a "**miss**" in cache.
- ✓ This technique ignores the CPU, concentrating on the interface between the cache and main memory.
- ✓ The **first-level** cache can be small enough to match the clock cycle time of the fast CPU.
- ✓ The **second-level** cache can be large enough to capture many accesses that would go to main memory, thereby lessening the effective miss penalty.
- ✓ **Local miss rate**—This rate is simply the number of misses in a cache divided by the total number of memory accesses to this cache.
- ✓ **Global miss rate**—The number of misses in the cache divided by the total number of memory accesses generated by the CPU

2. SECOND MISS PENALTY REDUCTION TECHNIQUE:

(CRITICAL WORD FIRST AND EARLY RESTART)

- ✓ **Critical word first**—Request the missed word first from memory and send it to the CPU as soon as it arrives; let the CPU continue execution while filling the rest of the words in the block.
- ✓ Critical-word-first fetch is also called fetch and requested word first
- ✓ **Early restart**—Fetch the words in normal order, but as soon as the requested word of the block arrives, send it to the CPU and let the CPU continue execution.

3. THIRD MISS PENALTY REDUCTION TECHNIQUE: (GIVING PRIORITY TO READ MISSES OVER WRITES)

- ✓ This optimization serves reads before writes have been completed.
- ✓ Write buffers, however, do complicate memory accesses in that they might hold the updated value of a location needed on a read miss.
- ✓ The alternative is to check the contents of the write buffer on a read miss, and if there are no conflicts and the memory system is available,
- ✓ The cost of writes by the processor in a write-back cache can also be reduced.
- ✓ Suppose a read miss will replace a dirty memory block. Instead of writing the dirty block to memory, and then reading memory, we could copy the dirty block to a buffer, then read memory, and for which the processor is probably waiting, will finish sooner.
- ✓ Similar to the situation above, if a read miss occurs, the processor can either stall until the buffers empty or check the addresses of the words in the buffer for conflicts.

4. FOURTH MISS PENALTY REDUCTION TECHNIQUE: (MERGING WRITE BUFFER)

- ✓ This technique also involves write buffers, this time improving their efficiency.
- ✓ Write through caches rely on write buffers, as all stores must be sent to the next lower level of the hierarchy.
- ✓ As mentioned above, even write back caches use a simple buffer when a block is replaced.
- ✓ If the write buffer is empty, the data and the full address are written in the buffer
- ✓ The write is finished from the CPU's perspective; the CPU continues working while the write buffer prepares to write the word to memory. If the buffer contains other modified blocks, the addresses can be checked to see if the address of this new data matches the address of the valid write buffer entry.
- ✓ If so, the new data are combined with that entry called **write merging**
- ✓ If the buffer is full and there is no address match, the cache (and CPU) must wait until the buffer has an empty entry.
- ✓ This optimization uses the memory more efficiently since multiword writes are usually faster than writes performed one word at time.

5. FIFTH MISS PENALTY REDUCTION TECHNIQUE: (VICTIM CACHES)

- ✓ One approach to lower miss penalty is to remember what was discarded in case it is needed again. Since the discarded data has already been fetched, it can be used again at small cost.
- ✓ Such recycling requires a small, fully associative cache between a cache and its refill path.
- ✓ This only block that are discarded from a cache because of a miss-victim-and are checked on a miss to see if they have the desired data before going to the next lower-level memory.
- ✓ If it is found there, the victim block and cache block are swapped.
- ✓ Depending on the program, a four-entry victim cache might remove one quarter of the misses in a 4-KB direct-mapped data cache.

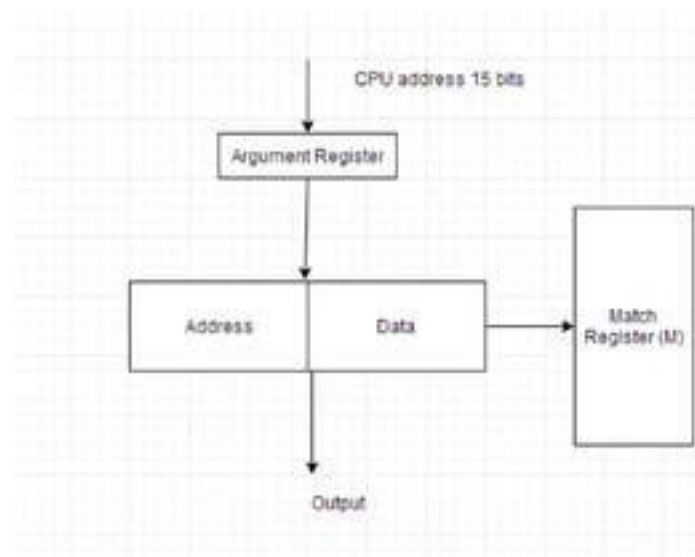
MEMORY MAPPING AND CONCEPT OF VIRTUAL MEMORY

The transformation of data from main memory to cache memory is called mapping.

There are 3 main types of mapping:

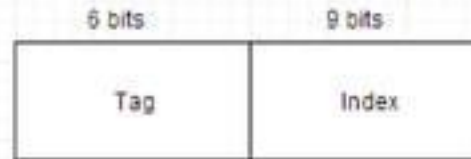
- Associative Mapping
- Direct Mapping
- Set Associative Mapping

ASSOCIATIVE MAPPING:



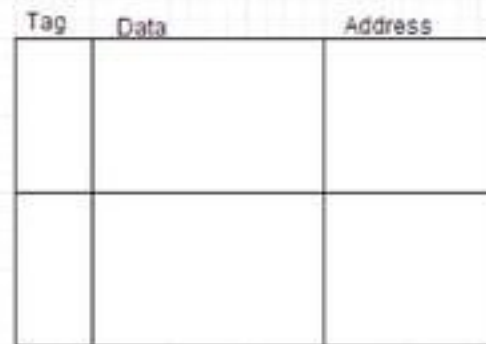
- ✓ The associative memory stores both address and data. The address value of 15 bits is 5-digit octal numbers and data is of 12 bits word in 4-digit octal number. A CPU address of 15 bits is placed in **argument register** and the associative memory is searched for matching address.

DIRECT MAPPING:



- ✓ The CPU address of 15 bits is divided into 2 fields. In this the 9 least significant bits constitute the **index** field and the remaining 6 bits constitute the **tag** field. The number of bits in index field is equal to the number of address bits required to access cache memory.

SET ASSOCIATIVE MAPPING:



- ✓ The disadvantage of direct mapping is that two words with same index address can't reside in cache memory at the same time. This problem can be overcome by set associative mapping.
- ✓ In this we can store two or more words of memory under the same index address.
- ✓ Each data word is stored together with its tag and this forms a set.

REPLACEMENT ALGORITHMS:

- ✓ Data is continuously replaced with new data in the cache memory using replacement algorithms.
- ✓ Following are the 2 replacement algorithms used:
 - FIFO - First in First out. Oldest item is replaced with the latest item.
 - LRU - Least Recently Used. Item which is least recently used by CPU is removed.

VIRTUAL MEMORY ORGANISATION

VIRTUAL MEMORY:

- ✓ Virtual memory is the separation of logical memory from physical memory.
- ✓ This separation provides large virtual memory for programmers when only small physical memory is available.
- ✓ Virtual memory is used to give programmers the illusion that they have a very large memory even though the computer has a small main memory.
- ✓ It makes the task of programming easier because the programmer no longer needs to worry about the amount of physical memory available.
- ✓ Virtual memory acts as a cache between main memory and secondary memory. Data is fetched in advance from the secondary memory (hard disk) into the main memory so that data is already available in the main memory when needed.
- ✓ The benefit is that the large access delays in reading data from hard disk are avoided.
- ✓ Pages are formulated in the secondary memory and brought into the main memory. This process is managed both in hardware (Memory Management Unit) and the software (The operating systems is responsible for managing the memory resources)
- ✓ The Memory Management unit (MMU) is located between the CPU and the physical memory. Each memory reference issued by the CPU is translated from the logical address space to the physical address space, guided by operating system-controlled mapping tables. As address translation is done for each memory reference, it must be performed by the hardware to speed up the process. The operating system is invoked to update the associated mapping tables.

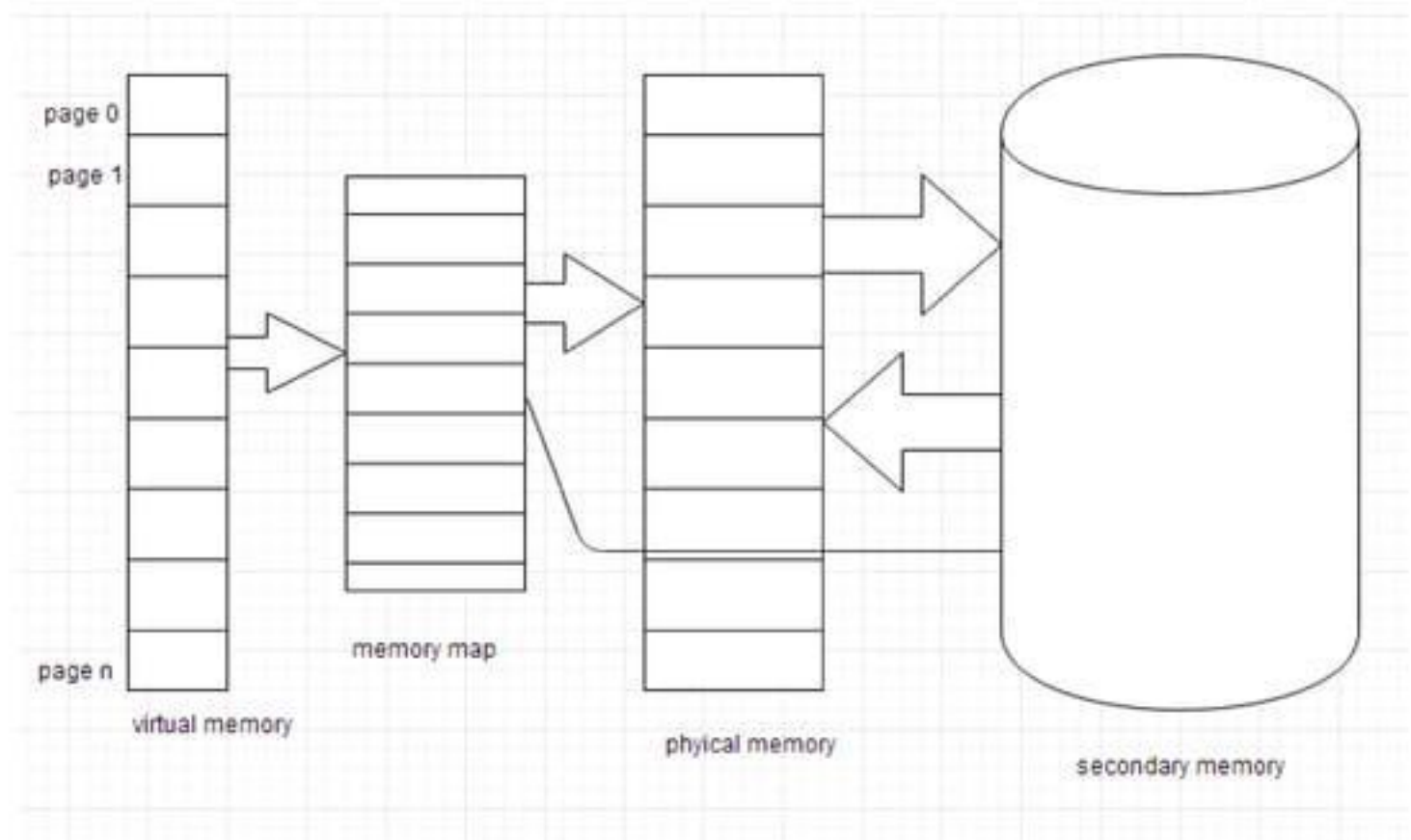
MEMORY MANAGEMENT AND ADDRESS TRANSLATION:

- ✓ The CPU generates the logical address. During program execution, effective address is generated which is an input to the MMU, which generates the virtual address.
- ✓ The virtual address is divided into two fields. First field represents the **page number** and the second field is the **word field**.
- ✓ In the next step, the MMU translates the virtual address into the physical address which indicates the location in the physical memory.

ADVANTAGES OF VIRTUAL MEMORY:

- ✓ Simplified **addressing scheme**: the programmer does not need to bother about the exact locations of variables/instructions in the physical memory. It is taken care of by the operating system.
- ✓ For a programmer, a large virtual memory will be available, even for a limited physical memory.
- ✓ Simplified access control.

VIRTUAL MEMORY ORGANIZATION



- ✓ Virtual memory can be organized in different ways. This first scheme is segmentation.

SEGMENTATION:

- ✓ In segmentation, memory is divided into segments of variable sizes depending upon the requirements.
- ✓ Main memory segments identified by segments numbers, start at virtual address 0, regardless of where they are located in physical memory.
- ✓ In pure segmented systems, segments are brought into the main memory from the secondary memory when needed. If segments are modified and not

required any more, they are sent back to secondary memory. This invariably results in gap between segments, called external fragmentation i.e. less efficient use of memory.

ADDRESSING OF SEGMENTED MEMORY

- ✓ The physical address is formed by adding each virtual address issued by the CPU to the contents of the segment base register in the MMU.
- ✓ Virtual address may also be compared with the segment limit register to keep track and avoiding the references beyond the specified limit. By maintaining table of segment base and limit registers, operating system can switch processes by switching the contents of the segment base and limit register.

PAGING:

- ✓ In this scheme, we have pages of fixed size. In demand paging, pages are available in secondary memory and are brought into the main memory when needed.
- ✓ Virtual addresses are formed by **concatenating** the **page number** with the **word number**.
- ✓ The MMU maps these pages to the pages in the physical memory and if not present in the physical memory, to the secondary memory.

PAGE SIZE:

- ✓ A very large page size results in increased access time. If page size is small, it may result in a large number of accesses.
- ✓ The main memory address is divided into 2 parts.
 - i. Page number: For virtual address, it is called virtual page number.
 - ii. Word Field

VIRTUAL ADDRESS TRANSLATION IN A PAGED MMU:

- ✓ Virtual address composed of a page number and a word number, is applied to the MMU. The virtual page number is limit checked to verify its availability within the limits given in the table. If it is available, it is added to the page table base address which results in a page table entry. If there is a limit check fault, a bound exception is raised as an interrupt to the processor.

PAGE TABLE:

- ✓ The page table entry for each page has two fields.
 - i. Page field
 - ii. Control Field: This includes the following bits.
 - **Access control bits:** These bits are used to specify read/write, and execute permissions.

- **Presence bits:** Indicates the availability of page in the main memory.
 - **Used bits:** These bits are set upon a read/ write.
- ✓ If the presence bit indicates a hit, then the page field of the page table entry contains the physical page number. It is concatenated with the word field of the virtual address to form a physical address.
 - ✓ Page fault occurs when a miss is indicated by the presence bit. In this case, the page field of the page table entry would contain the address of the page in the secondary memory.
 - ✓ Page miss results in an interrupt to the processor. The requesting process is suspended until the page is brought in the main memory by the interrupt service routine.
 - ✓ Dirty bit is set on a write hit CPU operation. And a write miss CPU operation causes the MMU to begin a write allocate process.

FRAGMENTATION:

- ✓ Paging scheme results in unavoidable internal fragmentations i.e. some pages may not be fully used. This results in wastage of memory.

PROCESSOR DISPATCH -MULTIPROGRAMMING:

- ✓ Consider the case, when a number of tasks are waiting for the CPU attention in a multiprogramming, shared memory environment. And a page fault occurs. Servicing the page fault involves these steps.
 - i. Save the state of suspended process
 - ii. Handle page fault
 - iii. Resume normal execution

SCHEDULING:

- ✓ If there are a number of memory interactions between main memory and secondary memory, a lot of CPU time is wasted in controlling these transfers and number of interrupts may occur. To avoid this situation, Direct Memory Access (DMA) is a frequently used technique.
- ✓ The Direct memory access scheme results in direct link between main memory and secondary memory, and direct data transfer without attention of the CPU.
- ✓ But use of DMA in virtual memory may cause coherence problem. Multiple copies of the same page may reside in main memory and secondary memory.
- ✓ The operating system has to ensure that multiple copies are consistent.

PAGE REPLACEMENT:

- ✓ On a page miss (page fault), the needed page must be brought in the main memory from the secondary memory. If all the pages in the main memory are being used, we need to replace one of them to bring in the needed page.
- ✓ Two methods can be used for page replacement.
 - i. **Random Replacement:** Randomly replacing any older page to bring in the desired page.
 - ii. **Least Frequently Used:** Maintain a log to see which particular page is least frequently used and to replace that page.
- ✓ **Translation Lookaside buffer:**
- ✓ Identifying a particular page in the virtual memory requires page tables (might be very large) resulting in large memory space to implement these page tables.
- ✓ To speed up the process of virtual address translation, translation Lookaside buffer (TLB) is implemented as a small cache inside the CPU, which stores the most recent page table entry reference made in the MMU. Its contents include
 - i. A mapping from virtual to physical address
 - ii. Status bits i.e. valid bit, dirty bit, protection bit
- ✓ It may be implemented using a fully associative organization
- ✓ **Operation of TLB:**
- ✓ For each virtual address reference, the TLB is searched associatively to find a match between the virtual page number of the memory reference and the virtual page number in the TLB. If a match is found (TLB hit) and if the corresponding valid bit and access control bits are set, then the physical page mapped to the virtual page is concatenated.

Working of Memory Sub System:

- ✓ When a virtual address is issued by the CPU, all components of the memory subsystem interact with each other. If the memory reference is a TLB hit, then the physical address is applied to the cache. On a cache hit, the data is accessed from the cache. Cache miss is processed.
- ✓ On a TLB miss (no match found) the page table is searched. On a page table hit, the physical address is generated, and TLB is updated and cache is searched. On a page table miss, desired page is accessed in the secondary
- ✓ memory, and main memory, cache and page table are updated. TLB is updated on the next access (cache access) to this virtual address

- ✓ To reduce the work load on the CPU and to efficiently use the memory sub system, different methods can be used. One method is separate cache for data and instructions.
- ✓ **Instruction Cache:** It can be implemented as a Translation Lookaside buffer.
- ✓ **Data Cache:** In data cache, to access a particular table entry, it can be implemented as a TLB either in the main memory, cache or the CPU

RAID (REDUNDANT ARRAYS OF INDEPENDENT DISKS)

RAID, or “**Redundant Arrays of Independent Disks**” is a technique which makes use of a combination of multiple disks instead of using a single disk for increased performance, data redundancy or both. The term was coined by David Patterson, Garth A. Gibson, and Randy Katz at the University of California, Berkeley in 1987.

Why data redundancy?

Data redundancy, although taking up extra space, adds to disk reliability. This means, in case of **disk failure**, if the same data is also backed up onto another disk, we can retrieve the data and go on with the operation. On the other hand, if the data is spread across just multiple disks without the RAID technique, the loss of a single disk can affect the entire data.

Key evaluation points for a RAID System

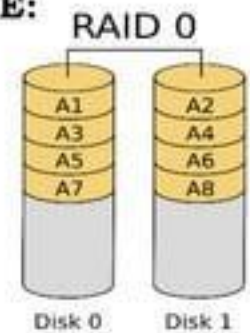
- **Reliability:** How many disk faults can the system tolerate?
- **Availability:** What fraction of the total session time is a system in uptime mode, i.e. how available is the system for actual use?
- **Performance:** How good is the response time? How high is the throughput (rate of processing work)? Note that performance contains a lot of parameters and not just the two.
- **Capacity:** Given a set of N disks each with B blocks, how much useful capacity is available to the user?

STANDARD RAID LEVELS:

RAID devices use many different architectures, called levels, depending on the desired balance between performance and **fault tolerance**. RAID levels describe how data is distributed across the drives. Standard RAID levels include the following:

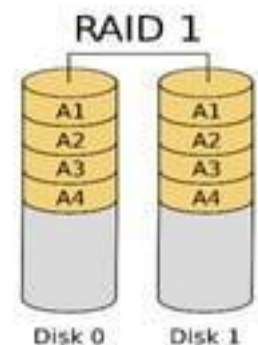
LEVEL 0: STRIPED DISK ARRAY WITHOUT FAULT TOLERANCE:

- ✓ Provides **data striping** (spreading out blocks of each file across multiple disk drives) but no redundancy.
- ✓ This improves performance but does not deliver fault tolerance. If one drive fails then all data in the array is lost.



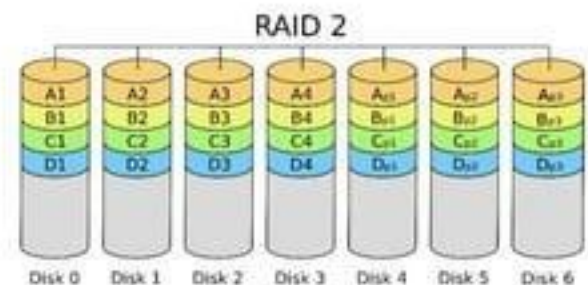
LEVEL 1: MIRRORING AND DUPLEXING:

- ✓ Provides disk mirroring. Level 1 provides twice the read transaction rate of single disks and the same write transaction rate as single disks.



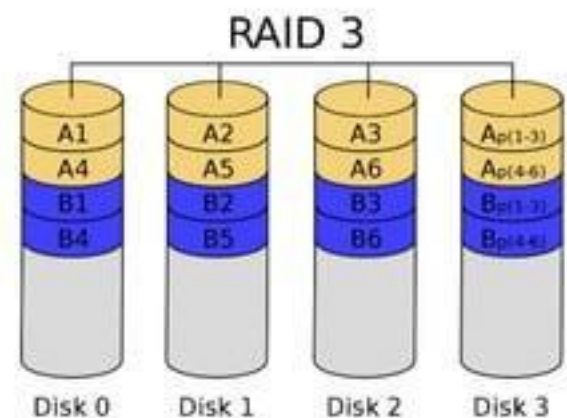
LEVEL 2: ERROR-CORRECTING CODING:

- ✓ Not a typical implementation and rarely used, Level 2 stripes data at the bit level rather than the block level.



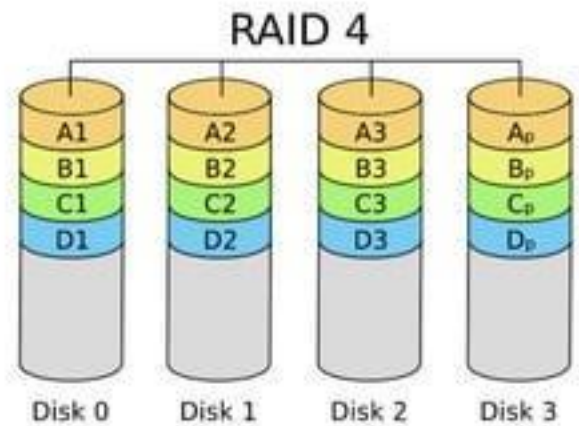
LEVEL 3: BIT-INTERLEAVED PARITY:

- ✓ Provides byte-level striping with a dedicated parity disk. Level 3, which cannot service simultaneous multiple requests, also is rarely used.



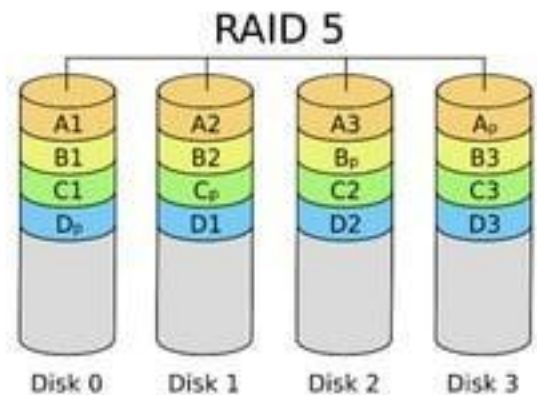
LEVEL 4: DEDICATED PARITY DRIVE:

- ✓ A commonly used implementation of RAID, Level 4 provides block-level striping (like Level 0) with a parity disk. If a data disk fails, the parity data is used to create a replacement disk. A disadvantage to Level 4 is that the parity disk can create write bottlenecks.



LEVEL 5: BLOCK INTERLEAVED DISTRIBUTED PARITY:

- ✓ Provides data striping at the byte level and also stripe error correction information. This results in excellent performance and good fault tolerance. Level 5 is one of the most popular implementations of RAID.



LEVEL 6: INDEPENDENT DATA DISKS WITH DOUBLE PARITY

- ✓ Provides block-level striping with parity data distributed across all disks.

